Algorithm 1: COUNTOPTIMAANDBACKTRACK for Interval Scheduling

Input: Intervals with IDs (i, s_i, f_i), $i = 1, \ldots, m$.
Output: Optimal size $k^\star$, number of optimal schedules $\mathrm{cnt}[n]$, unique set $S^\star$ if $\mathrm{cnt}[n] = 1$.
Sort intervals by (f_i, s_i, i) to obtain $J = (1, \ldots, n)$; precompute $p(i) = \max\{j < i : f_j \le s_i\}$ by binary search
$\mathrm{opt}[0] \leftarrow 0$, $\mathrm{cnt}[0] \leftarrow 1$
for $i \leftarrow 1$ **to** n **do**
  INCL $\leftarrow 1 + \mathrm{opt}[p(i)]$, EXCL $\leftarrow \mathrm{opt}[i-1]$
  $\mathrm{opt}[i] \leftarrow \max\{\text{INCL}, \text{EXCL}\}$
  if INCL $>$ EXCL **then** $\mathrm{cnt}[i] \leftarrow \mathrm{cnt}[p(i)]$
  else if EXCL $>$ INCL **then** $\mathrm{cnt}[i] \leftarrow \mathrm{cnt}[i-1]$
  else $\mathrm{cnt}[i] \leftarrow \mathrm{cnt}[p(i)] + \mathrm{cnt}[i-1]$
$k^\star \leftarrow \mathrm{opt}[n]$
if $\mathrm{cnt}[n] \neq 1$ **then return** $(k^\star, \mathrm{cnt}[n], \emptyset)$
`// Uniqueness holds: recover the single optimal set`
$S^\star \leftarrow \emptyset$; $i \leftarrow n$
while $i > 0$ **do**
  INCL $\leftarrow 1 + \mathrm{opt}[p(i)]$, EXCL $\leftarrow \mathrm{opt}[i-1]$
  if INCL $>$ EXCL **then**
    add i to $S^\star$; $i \leftarrow p(i)$
  else if EXCL $>$ INCL **then**
    $i \leftarrow i - 1$
  else
    if $\mathrm{cnt}[p(i)] = 1$ *and* $\mathrm{cnt}[i-1] = 0$ **then**
      add i to $S^\star$; $i \leftarrow p(i)$
    else
      $i \leftarrow i - 1$
return $(k^\star, 1, S^\star)$ `// Sort` $S^\star$ `by` (f, i) `when reporting.`

values. The canonical target we release per instance is: (i) the unique LIS *index* sequence listed in increasing row order as `\ids{...}`, and (ii) its length L as `\answer{L}`. *IDs are 1-based row indices*.

Sampling. We work on an integer grid. For each trial we: (i) draw a length $m \sim \mathrm{Unif}\{m_{\min}, \ldots, m_{\max}\}$; (ii) draw values i.i.d. $a_i \sim \mathrm{Unif}\{V_{\min}, \ldots, V_{\max}\}$; (iii) accept the instance only if the LIS is *unique* (by index sequence) and $L \ge 2$. Typical bounds used in our experiments are $m_{\min}$=5, $m_{\max}$=16, $V_{\min}$=1, $V_{\max}$=1000, but any fixed choices are valid.

Uniqueness check by $O(n^2)$ counting. Let $\texttt{len_end}[i]$ be the LIS length ending at i, and $\texttt{cnt_end}[i]$ the number of LIS that end at i with that maximum length. Initialize $\texttt{len_end}[i]$=1, $\texttt{cnt_end}[i]$=1. For each $i = 1..n$ and $j < i$:

$$\text{if } a_j < a_i : \quad \begin{cases} \text{if } \texttt{len_end}[j]{+}1 > \texttt{len_end}[i] : & \texttt{len_end}[i] \leftarrow \texttt{len_end}[j]{+}1,\ \texttt{cnt_end}[i] \leftarrow \texttt{cnt_end}[j] \\ \text{else if } \texttt{len_end}[j]{+}1 = \texttt{len_end}[i] : & \texttt{cnt_end}[i] \leftarrow \texttt{cnt_end}[i] + \texttt{cnt_end}[j]. \end{cases}$$

Let $L = \max_i \texttt{len_end}[i]$ and $\#\mathrm{LIS} = \sum_{i:\texttt{len_end}[i]=L} \texttt{cnt_end}[i]$. We declare *uniqueness* iff $\#\mathrm{LIS} = 1$.

Canonical reconstruction (used only after uniqueness holds). We reconstruct one LIS index sequence via patience sorting with predecessor links: scan left to right; for each a_i place it by lower_bound (first position $\ge a_i$) in the tails array; store for each i a predecessor pointer to the last index at the previous length; then backtrack from the final tail index to obtain indices in increasing order.

Algorithm 2: GENERATEUNIQUEACTIVITYINSTANCE

Input: RNG seed; $m_{\min}, m_{\max}$; minute-grid bounds $S_{\max}$, $D_{\min}, D_{\max}$; `max_tries`.
Output: Intervals $\{(i, s_i, f_i)\}_{i=1}^{m}$; canonical `\ids{}` sequence; integer answer $|S^\star|$.

```
for t ← 1 to max_tries do
    Sample m ~ Unif{m_min, ..., m_max}
    I ← ∅
    while |I| < m do
        sample s ~ Unif{0, ..., S_max}, d ~ Unif{D_min, ..., D_max}, set f ← s + d
        if f ≤ S_max + D_max then append new interval (|I|+1, s, f) to I
    (k*, cnt, S*) ← CountOptimaAndBacktrack(I)
    if cnt ≠ 1 then continue
    G ← greedy earliest-finish schedule on I using (f, s, i) tie-breaks
    if G ≠ S* then continue
    // Canonical target: IDs of S* sorted by (f, i); answer = |S*|
    return (I, \ids{ IDs(S*) }, \answer{ |S*| })
Fail if no instance is found within max_tries (try another seed or bounds)
```

Algorithm 3: COUNTLISLENGTHANDNUMBER (strict LIS length & count)

Input: Sequence $(a_1, \ldots, a_n) \in \mathbb{Z}^n$.
Output: L (LIS length), #LIS (number of LIS).

```
for i ← 1 to n do
    len_end[i] ← 1;
    cnt_end[i] ← 1
for i ← 1 to n do
    for j ← 1 to i−1 do
        if a_j < a_i then
            if len_end[j]+1 > len_end[i] then
                len_end[i] ← len_end[j]+1;
                cnt_end[i] ← cnt_end[j]
            else if len_end[j]+1 = len_end[i] then
                cnt_end[i] ← cnt_end[i] + cnt_end[j]
L ← max_i len_end[i];   #LIS ← Σ_{i:len_end[i]=L} cnt_end[i]
return (L, #LIS)                                   // Runtime O(n^2)
```

Canonical reporting and prompts. Because the LIS is a subsequence, the ground-truth `\ids{...}` is simply the unique LIS indices $(i_1 < \cdots < i_L)$ in increasing row order; `\answer{L}` reports its length.

Complexity. Per trial: $O(n^2)$ for counting uniqueness; $O(n \log n)$ to reconstruct the indices once unique. Overall generation is bounded by `max_tries`, which we set large enough so failures are rare.

B Extracting sorted ID lists from free-form responses

Setup. For each Activity instance the prompt shows an ASCII table with unique integer IDs $i \in \{1, \ldots, n\}$ and times (s_i, f_i). We define the ground-truth sorted order

$$B = (b_1, \ldots, b_n) \;:=\; \text{IDs sorted by } (f_i, i) \text{ (non-decreasing end time, ties by smaller ID).}$$

This B is the order used by the greedy earliest-finish-time scheduler (Alg. 1, §3.1). From the model's free-form reply r (arbitrary text) we attempt to recover a list the model claims to be "sorted."

Candidate sources (lightweight and order-preserving). We build up to four candidate ID sequences from r; all candidates are *normalized* by (i) filtering to the valid ID set $\mathcal{I} = \{1, \ldots, n\}$

Algorithm 4: GENERATEUNIQUELISINSTANCE

Input: RNG seed; $m_{\min}, m_{\max}$; value bounds $V_{\min}, V_{\max}$; `max_tries`.
Output: Values $(a_1, \ldots, a_m)$; canonical `\ids{·}`; `\answer{·}`.
for $t \leftarrow 1$ **to** *`max_tries`* **do**
  Sample $m \sim \mathrm{Unif}\{m_{\min}, \ldots, m_{\max}\}$; Sample $a_i \sim \mathrm{Unif}\{V_{\min}, \ldots, V_{\max}\}$ i.i.d. for i=1..m
  $(L, \#) \leftarrow$ COUNTLISLENGTHANDNUMBER$(a_{1:m})$
  if $L < 2$ ***or*** $\# \neq 1$ **then continue**
  `// Reconstruct unique LIS indices via patience sorting with predecessors`
  $(i_1, \ldots, i_L) \leftarrow$ PATIENCERECONSTRUCT$(a_{1:m})$ **return** $\big(a_{1:m},$ *`\ids`*$\{i_1, \ldots, i_L\},$ *`\answer`*$\{L\}\big)$
Fail if no unique instance within `max_tries` (try new seed or bounds)

Activity Scheduling Prompt

Determine the largest subset of activities that can be scheduled without any overlaps (a single resource is available, so no double-booking).

ID	Start	End
1	06:09	07:24
2	07:13	08:23
3	07:29	09:28
4	08:24	10:18
5	04:48	06:14

Instructions:

- Times are given in 24-hour HH:MM format.
- Non-overlap means an activity ending at time T is compatible with one starting at T.
- Select a maximum-size subset of non-overlapping rows.
- Your output must include the following two lines at the end, in this exact format:
  1. `\ids{<comma-separated IDs of the chosen rows, listed in order of increasing END time. If two rows have the same END time, put the smaller ID first>}`
  2. `\answer{<number of chosen rows>}`
- No spaces inside `\ids{...}`. Example: `\ids{3,9,12}`

Hint: Sort the rows by increasing end time, then greedily pick compatible rows.

Ground truth: `\ids{5,2,4}`, `\answer{3}`

LIS Prompt

Determine the longest strictly increasing subsequence (by VALUE) from the rows below (use the row IDs).

ID	Value
1	797
2	476
3	335
4	452
5	606

Instructions:

- The table lists 1-based row IDs and integer values.
- A valid subsequence must be strictly increasing by VALUE and preserve the original row order.
- Select a maximum-size subsequence.
- Your output must include the following two lines at the end, in this exact format:
  1. `\ids{<comma-separated IDs of the chosen rows, listed in increasing ROW order>}`
  2. `\answer{<number of chosen rows>}`
- No spaces inside `\ids{...}`. Example: `\ids{3,9,12}`

Hint: Use LIS DP (len[i] = 1 + $\max$ len[j] for $j < i$ with value[j]<value[i]) while storing prev[i] (or patience sorting with predecessor links); then backtrack to output the ids.

Ground truth: `\ids{3,4,5}`, `\answer{3}`

Figure 6: Example prompts and ground-truth for Activity Scheduling (left) and LIS (right).

derived from the prompt table, and (ii) de-duplicating while preserving the *first* occurrence of each ID in the text.

1. **Sorted-block (for exact-sorting only).** We split r into paragraphs and select those that mention a sorting token ("sort/sorted/sorting"). To isolate the purported sorting step, we truncate at the first stop word (any of: `select, greedy, choose, subset, largest, final answer, so, thus, therefore, next`). We then extract IDs from this truncated text via either "`ID` k" tokens or the longest comma-separated integer run. If the resulting list contains *all* n distinct IDs exactly once, we accept it as a full "sorted-block" candidate A_{full} and reserve it solely for the exact-sorting check below.
2. `\ids{...}` **blocks.** From every brace block we read the comma-separated integers, normalize, and keep the resulting sequence if non-empty.